技术对接文档v20250816

- 技术对接文档v20250816
  - 提前准备项
  - 主要流程
  - 生成RSA密钥对和配置商户公钥
    - 第一种方式(商户公钥会自动保存到平台，不需要商户再去配置) 登录商户后台 在【商户配置】-【开发者信息】找到【一键生成商户公私钥】按钮，点击之后会自动生成商户公私钥对，商户只需要拷贝对应开发语言私钥即可（注：本平台只提供了 JAVA/PHP这两种语言的RSA公私钥对的生成方式，其他开发语言请参考下面第二种方式生成）
    - 第二种方式(商户公钥需要手动配置到商户后台) Linux用户(以Mac为例)
      - 1.生成商户RSA私钥（必选）
      - 2.把商户RSA私钥转换成PKCS8格式（可选）
      - 3.生成商户RSA公钥（必选）
  - 签名/验签
    - 请求数据签名
      - 组装参数
      - 进行编码
    - 响应数据验签
      - 组装参数
      - 进行编码
      - 进行验签
    - 注意事项
      - RSA签名/验签方法代码示例
      - java语言
      - php语言
  - 下单
      - 请求参数
      - 返回结果
      - 下单示例
      - 请求body报文(json格式)
      - 响应body报文(json格式)
  - 查询订单
      - 请求参数
      - 返回结果
      - 查询订单示例
      - 请求body报文(json格式)
      - 响应body报文(json格式)
  - 对账单下载
    - 对账单下载
      - 请求参数
      - 返回结果
        - 失败时返回以下字段
        - 成功时返回信息如下
      - 示例
        - 交易对账单成功示例
  - 后台异步通知
    - 后台异步通知
    - 如何验签

可靠通知规则
支付平台请求参数说明
通知报文内容示例
支付成功通知示例
商户端返回结果
错误码
注意事项

提前准备项

- 申请支付平台账号
- 申请开通支付产品功能
- 对接联调测试

主要流程

商户通过调用 **下单**、**查询订单**、**对账单下载**、**后台异步通知**等接口完成**支付**功能的对接

生成RSA密钥对和配置商户公钥

第一种方式(商户公钥会自动保存到平台，不需要商户再去配置) 登录商户后台 在【商户配置】-【开发者信息】找到【一键生成商户公私钥】按钮，点击之后会自动生成商户公私钥对，商户只需要拷贝对应开发语言私钥即可（注：本平台只提供了 JAVA/PHP这两种语言的RSA公私钥对的生成方式，其他开发语言请参考下面第二种方式生成）

第二种方式(商户公钥需要手动配置到商户后台) Linux用户(以Mac为例)

1.生成商户RSA私钥（必选）

```
#在Terminal中输入下面命令（该命令会生成2048位的私钥）：
openssl genrsa -out rsa_private_key.pem 2048
#回车后如下显示：
Generating RSA private key, 2048 bit long modulus
...++++++
........++++++
e is 65537 (0x10001)
#此时你可以在当前目录下看到rsa_private_key.pem文件了
```

2.把商户RSA私钥转换成PKCS8格式（可选）

- 对于使用Java语言的开发者,将pkcs8在console中输出的私钥去除头尾、换行和空格,作为开发者私钥,对于.NET和PHP的开发者来说,无需进行pkcs8命令行操作.

```
#在Terminal中输入下面命令:
openssl pkcs8 -topk8 -inform PEM -in rsa_private_key.pem -outform PEM -
nocrypt -out rsa_private_key_pkcs8.pem
#回车后生成PKCS8格式的私钥(rsa_private_key_pkcs8.pem).
```

3.生成商户RSA公钥（必选）

- 往 商户后台-【商户配置】-【开发者信息】-商户公钥输入框处 保存商户公钥内容之前,务必需要手动删除商户RSA公钥内容的头尾、换行和空格

```
#在Terminal中输入下面命令:
openssl rsa -in rsa_private_key.pem -out rsa_public_key.pem -pubout
#回车后如下所示:
writing RSA key
#此时你可以看到一个文件名为rsa_public_key.pem的文件,这个就是公钥.
```

签名/验签

- https传输协议
- 采用双向RSA签名
  - 商户在发送请求时,需要使用商户私钥将请求数据进行签名（必选）
  - 商户在接收到返回值时,需要使用支付平台公钥将返回数据中的签名进行校验（可选，建议为了安全，务必对响应数据验签）
- 公钥交换
  - 商户在商户后台设置的商户公钥,支付平台使用该商户公钥验签商户请求
  - 支付平台公钥可以在商户后台“商户配置”-“开发者信息”-“支付平台公钥”查看,用于验签支付平台的响应数据

请求数据签名

参与签名的字段及顺序:

顺序	变量名	获取来源	描述	示例
1	method	http_method	请求方法,小写(post/get)	固定：post
2	query_string	url	查询字符串	固定（空字符串）：""
3	nonce	header	8~128位的随机字符串,由字母数字组成,不允许特殊字符	0d12ef2a0942485cb6ee90e2f6a8dde6
4	timestamp	header	13位的时间戳,标准北京时间,时区为东八区,自1970年1月1日 0点0分0秒 以来的毫秒数	1749873643180
5	Authorization	header	商户secretKey,请到商户后台点击“商户配置”-“开发者信息”-“SecretKey”查看	31a82ecd6a414205bba3f5959787ec9b
6	request_data	body	请求参数对象,JSON格式	{"product_desc":"Recebimento de 10 menos 1","user_ip":"111.11.32.32","amount":"1100","time_start":"20250614120042","description":"b

签名步骤:

组装参数

- 以 换行符（"\n"） 拼接各个字段 注意: query_string 数值为空 默认为空字符串即可

```
method+"\n"+query_string+"\n"+nonce+"\n"+timestamp+"\n"+Authorization+"\n"+request_data
```

不同的语言,换行符可能有所不同

组装成要签名的数据,例如:

```
post

0d12ef2a0942485cb6ee90e2f6a8dde6
1749873643180
31a82ecd6a414205bba3f5959787ec9b
{"product_desc":"Recebimento de 10 menos 1","user_ip":"111.11.32.32","amount":"1100","time_start":"20250614120042","description":"beneficiário","trade_type":"pix","notify_url":"https://merchant.notify.com","merchant_order_no":"M1de76cc356684c3cbc5ac59562c5bb48","product_title":"beneficiário"}
```

进行编码

- 以UTF-8编码将待签名字符串转换成字节数组,然后使用**商户私钥**对其签名,签名算法为SHA256WithRSA,将签名后的结果以Base64编码转码后存放到header中,key为'sign'.

假如签名结果为:

```
HHYnZ95kceS+ccyYr8lgglWLG8dnO7w3DOZ1Y4x9zh2DoYYVMRoGz/666eLKltHDXrRF1P0TDlMs+
ywYamz+cSWRIDw1HARF7dwTfPvQknasJyGXztraEXhppo/y306/yxzdu1YKCgyQiDVlaWoMCJig8g
AjSBWcCoIKxGAD4PsRfUplB2B0EhckC3xuJgbmdc88OMaA2E3HIHW7WAmycFGMGKCP9TqVDWdlHJ4
aCgNNNwDbD7TY3hcU+lrME+7om6ofB7+hs0Rv3qGiioUbs9YS/+C1jJucQ5HB6NY+c3NtWNvQU1Td
M/8f2K1aerbcyE7wedeCZ0WIEZdDbty0IQ==
```

则最后的请求Header是:

```
POST /v1/order/create HTTP/1.1
Host: 替换支付系统域名地址/merchant-api
ContentType: application/json;charset=utf-8
Authorization: 31a82ecd6a414205bba3f5959787ec9b
nonce: 0d12ef2a0942485cb6ee90e2f6a8dde6
timestamp: 1749873643180
sign:
HHYnZ95kceS+ccyYr8lgglWLG8dnO7w3DOZ1Y4x9zh2DoYYVMRoGz/666eLKltHDXrRF1P0TDlMs+
ywYamz+cSWRIDw1HARF7dwTfPvQknasJyGXztraEXhppo/y306/yxzdu1YKCgyQiDVlaWoMCJig8g
AjSBWcCoIKxGAD4PsRfUplB2B0EhckC3xuJgbmdc88OMaA2E3HIHW7WAmycFGMGKCP9TqVDWdlHJ4
aCgNNNwDbD7TY3hcU+lrME+7om6ofB7+hs0Rv3qGiioUbs9YS/+C1jJucQ5HB6NY+c3NtWNvQU1Td
M/8f2K1aerbcyE7wedeCZ0WIEZdDbty0IQ==

{"product_desc":"Recebimento de 10 menos
1","user_ip":"111.11.32.32","amount":"1100","time_start":"20250614120042","de
scription":"beneficiário","trade_type":"pix","notify_url":"https://merchant.n
otify.com","merchant_order_no":"M1de76cc356684c3cbc5ac59562c5bb48","product_t
itle":"beneficiário"}
```

响应数据验签

参与验签的字段及顺序:

顺序	参数	获取位置	描述	示例
1	nonce	header	8~128位的随机字符串,由字母数字组成,不允许特殊字符	77865cf153f74a10bd75398772236c72
2	timestamp	header	13位的时间戳,标准北京时间,时区为东八区,自1970年1月1日 0点0分0秒 以来的毫秒数	1594630395263
3	Authorization	header	商户SecretKey,请到商户后台点击“商户配置”-“开发者信息”-“SecretKey”查看	500072314e1e46bbb32b04f85f6dfbfa
4	response_data	body	响应数据对象,JSON格式	{"credential":{"pix":"00020101021226900014br.gov.bcb.pix2568qrcode.siliumpay.com.br/dynam Paulo62070503***6304F505"},"merchant_order_no":"M1de76cc356684c3cbc5ac59562c5bb48",

验签步骤:

返回的response示例:

```
ContentType: application/json;charset=utf-8
Authorization: 31a82ecd6a414205bba3f5959787ec9b
nonce: 974830e8e50c4b2e9a97dcbe926a03f9
timestamp: 1749873643992
sign:
lx+eLiAb7unskt32uBM/Jom64AnYFCReayDRHI/EACwGCPPrFp1AdciVwelx+zVMmQGHOnBoXHHXT
e9j4f0++HkS05dqf5SS5TE3DMAJsx7hmOsDVAGvGNFnryxBo8Gm+Woi0TYatQpsJBjk7Si3mHicla
XW37SvzkxKXHm35h68vKvxmliv/ypiZWK+xgv4nRbFkI3J/ZaHY+IZWD7wZ1YaeXw3ftsxPfV9bL7
c5OR+46QAnrTjNASVQVKbSAzAzQlYLtbgtykhahk+qisRda44NevqRp8zwRxshM1P/LRfhkgSaBls
xhm5kHHVhcks+2h7yj7ImONhGmT7QYt0sw==


{"credential":
{"pix":"00020101021226900014br.gov.bcb.pix2568qrcode.siliumpay.com.br/dynamic
/7f47b3e5-b5d1-49a2-b0e7-eb76105cae46520400005303986580 2BR5907RW LTDA6009Sao
Paulo62070503***6304F505"},"merchant_order_no":"M1de76cc356684c3cbc5ac59562c5
bb48","order_no":"TD250614138341585927556300 8","return_code":"SUCCESS","retur
n_msg":"OK","status":"SUCCESS","trade_type":"pix"}
```

组装参数

- 验签前以 '\n' 拼接各个字段:

```
nonce+'\n'+timestamp+'\n'+Authorization+'\n'+response_data
```

组装成要验签的数据,例如:

```
974830e8e50c4b2e9a97dcbe926a03f9
1749873643992
31a82ecd6a414205bba3f5959787ec9b
{"credential":
{"pix":"00020101021226900014br.gov.bcb.pix2568qrcode.siliumpay.com.br/dynamic
/7f47b3e5-b5d1-49a2-b0e7-eb76105cae46520400005303986580 2BR5907RW LTDA6009Sao
Paulo62070503***6304F505"},"merchant_order_no":"M1de76cc356684c3cbc5ac59562c5
bb48","order_no":"TD25061413834158592755630 08","return_code":"SUCCESS","retur
n_msg":"OK","status":"SUCCESS","trade_type":"pix"}
```

进行编码

- 以UTF-8编码将待验签字符串转换成字节数组,然后使用**支付平台公钥**对其签名,签名算法为SHA256WithRSA.

进行验签

- 将第二步的签名结果和Header中取到的sign进行比对,相同即为验签通过,可以说明该响应数据来自于支付平台.

注意事项

- 对数据进行签名或验签时,字段顺序不能变;
- 对签名后的值要进行Base64编码后才能往header中存放;
- 验签时对获取到的签名数据【sign】 要进行 Base64 解码后再进行验证;
- 验签时要对 签名数据、商户secret key、请求的时效性 进行验证.

RSA签名/验签方法代码示例

java语言

```
import javax.crypto.Cipher;
import java.security.KeyFactory;
import java.security.PrivateKey;
import java.security.PublicKey;
import java.security.spec.PKCS8EncodedKeySpec;
import java.security.spec.X509EncodedKeySpec;


public class RSA {

    public static String ALGORITHM = "RSA";
    public static String SIGN_ALGORITHMS = "SHA256WithRSA";// 摘要加密算饭
    public static String CHAR_SET = "UTF-8";
```

```
    /**
     * 数据签名
     *
     * @param content
     *            签名内容
     * @param privateKey
     *            私钥
     * @return 返回签名数据
     */
    public static String sign(String content, String privateKey) {
        try {
            PKCS8EncodedKeySpec priPKCS8 = new PKCS8EncodedKeySpec(
                    Base64.decode(privateKey));
            KeyFactory keyf = KeyFactory.getInstance("RSA");
            PrivateKey priKey = keyf.generatePrivate(priPKCS8);
            java.security.Signature signature = java.security.Signature
                    .getInstance(SIGN_ALGORITHMS);
            signature.initSign(priKey);
            signature.update(content.getBytes(CHAR_SET));
            byte[] signed = signature.sign();
            return Base64.encode(signed);
        } catch (Exception e) {
            e.printStackTrace();
        }
        return null;
    }

    /**
     * 签名验证
     *
     * @param content
     * @param sign
     * @param public_key
     * @return
     */
    public static boolean verify(String content, String sign,String
public_key) {
        try {
            KeyFactory keyFactory = KeyFactory.getInstance("RSA");
            byte[] encodedKey = Base64.decode(public_key);
            PublicKey pubKey = keyFactory
                    .generatePublic(new X509EncodedKeySpec(encodedKey));
            java.security.Signature signature = java.security.Signature
                    .getInstance(SIGN_ALGORITHMS);
            signature.initVerify(pubKey);
            signature.update(content.getBytes(CHAR_SET));
            return signature.verify(Base64.decode(sign));
        } catch (Exception e) {
            e.printStackTrace();
        }
        return false;
    }
```

php语言

```
/**
     * 签名数据
     * @param $data 待签名数据
     * @param $privateKeyPath 商户私钥文件路径
     * @return string 签名后的数据
     */
    public static function sign($data, $privateKeyPath)
    {
        //读取私钥文件
        $priKey = file_get_contents($privateKeyPath);
        //转换为openssl密钥，必须是没有经过pkcs8转换的私钥
        $res = openssl_get_privatekey($priKey);
        //调用openssl内置签名方法，生成签名$sign
        openssl_sign($data, $sign, $res,OPENSSL_ALGO_SHA256);
        //释放资源
        openssl_free_key($res);
        //base64编码
        return base64_encode($sign);
    }

    /**
     * 验签数据
     * @param $data 原始数据
     * @param $sign 签名数据
     * @param $platformPublicKeyPath 支付平台公钥文件
     * @return bool 验签结果
     */
    public static function verify($data, $sign, $platformPublicKeyPath)  {
        //读取支付平台公钥文件
        $pubKey = file_get_contents($platformPublicKeyPath);
        //转换为openssl格式密钥
        $res = openssl_get_publickey($pubKey);
        //调用openssl内置方法验签，返回bool值
        $result = (bool)openssl_verify($data,base64_decode($sign),
$res,OPENSSL_ALGO_SHA256);
        //释放资源
        openssl_free_key($res);
        //返回资源是否成功
        return $result;
    }
```

下单

使用HTTP协议调用下单接口发起请求：

```
POST /v1/order/create HTTP/1.1
Host: 替换支付系统域名地址/merchant-api
ContentType: application/json;charset=utf-8
Authorization: {开发者SecretKey}
nonce: {随机字符串}
timestamp: {时间戳}
sign: {签名数据}
```

请求参数

字段名	变量名	类型	必填	示例值	描述
商户订单号	merchant_order_no	String(64)	是	M1de76cc356684c3cbc5ac59562c5bb48	商户系统内部订单号，要求64个字符内，只能是数字、大小写字母_-
商品标题	product_title	String	是	beneficiário	购买商品的标题
商品描述	product_desc	String(128)	是	beneficiário	购买商品的描述信息，要求128个字符内
金额	amount	Int	是	1000	订单总金额，单位为分
订单描述	description	String(300)	否	beneficiário	订单描述，要求300个字符内，不能包含特殊字符:英文逗号，注意：下载对账单会返回此字段数据
交易类型	trade_type	String(30)	是	pix	交易类型:请联系客服获取
接口异步通知地址URL	notify_url	String(500)	是	http://wwww1344.com/trade/notify	支付结果异步通知地址，HTTP/HTTPS开头字符串，注意：订单状态成功才会通知该地址
用户下单IP	user_ip	String	是	123.11.23.13	用户下单实际IP地址
交易创建时间	time_start	String(14)	是	20190918112605	订单生成时间，格式为yyyyMMddHHmmss

返回结果

字段名	变量名	类型	必填	示例值
网关返回码	return_code	String(16)	是	SUCCESS
网关返回码描述	return_msg	String(128)	是	OK
商户订单号	merchant_order_no	String(64)	否	M1de76cc356684c3cbc5ac59562c5bb48
支付平台订单号	order_no	String(29)	否	TD25061413834158592755630 08
订单金额	amount	Int	否	1000
交易类型	trade_type	String(30)	否	pix
支付凭据	credential	String	否	pix支付返回信息 {"pix":"00020101021226900014br.gov.bcb.pix2568qrcode.siliumpay.com.br/dynamic/7f47b3b5d1-49a2-b0e7-eb76105cae4652040000530398658 02BR5907RW LTDA6009Sao Paulo62070503***6304F505","cashierUrl":"H5支付链接地址URL"}
订单受理状态	status	String(16)	否	PROCESSING

下单示例

请求body报文(json格式)

```
{
    "product_desc": "Recebimento de 10 menos 1",
    "user_ip": "111.11.32.32",
    "amount": "1100",
    "time_start": "20250614120042",
    "description": "beneficiário",
    "trade_type": "pix",
    "notify_url": "https://merchant.notify.com",
    "merchant_order_no": "M1de76cc356684c3cbc5ac59562c5bb48",
    "product_title": "beneficiário"
}
```

响应body报文(json格式)

```
{
    "credential": {
        "pix":
"00020101021226900014br.gov.bcb.pix2568qrcode.siliumpay.com.br/dynamic/7f47b3
e5-b5d1-49a2-b0e7-eb76105cae46520400005303986580 2BR5907RW LTDA6009Sao
Paulo62070503***6304F505",
        "cashierUrl":"https://api.xx.com/cashier/pay.html?
orderNo%3DTD250706139152109366747545 6%26merchantOrderNo%3DM4ccf039abd9f4ee18c
a538523b513d4b%26orderAmount%3D11.00%26payType%3Dpix%26ts%3D1751806082341%26o
riginalFlag%3D0%26encryptOrderNo%3De88c436c3faa2a4adbe973c95b20d923"
    },
    "merchant_order_no": "M1de76cc356684c3cbc5ac59562c5bb48",
    "order_no": "TD250614138341585927556300 8",
    "return_code": "SUCCESS",
    "return_msg": "OK",
    "status": "SUCCESS",
    "trade_type": "pix"
}
```

查询订单

```
POST /v1/order/query HTTP/1.1
Host: 替换支付系统域名地址/merchant-api
ContentType: application/json;charset=utf-8
Authorization: {开发者SecretKey}
nonce: {随机字符串}
timestamp: {时间戳}
sign: {签名数据}
```

请求参数

字段名	变量名	类型	必填	示例值	描述
支付平台订单号	order_no	String(29)	二选一	TD250614138341585927556300 8	支付平台订单号
商户订单号	merchant_order_no	String(64)		M1de76cc356684c3cbc5ac59562c5bb48	商户订单号

返回结果

字段名	变量名	类型	必填	示例值
网关返回码	return_code	String(16)	是	SUCCESS
网关返回码描述	return_msg	String(128)	是	OK
商户订单号	merchant_order_no	String(64)	否	M1de76cc356684c3cbc5ac59562c5bb48
支付平台订单号	order_no	String(29)	否	TD25061413834158592755630 08
银行流水E单号	endToEndId	String	否	E250614138341563008
交易类型	trade_type	String(30)	否	pix
支付凭据	credential	String	否	返回信息 {"pix":"00020101021226900014br.gov.bcb.pix2568qrcode.siliumpay.com.br/dynamic/7f47b3b5d1-49a2-b0e7-eb76105cae4652040000530398658 02BR5907RW LTDA6009Sao Paulo62070503***6304F505"}
订单状态	status	String(16)	否	SUCCESS
订单金额	amount	Int	否	1000
手续费金额	fee	Int	否	30

查询订单示例

请求body报文(json格式)

```
{
    "order_no": "TD250614138341585927556300 8",
    "merchant_order_no": ""
}
```

响应body报文(json格式)

```
{
    "amount": 1100,
    "credential": {
        "pix":
"00020101021226900014br.gov.bcb.pix2568qrcode.siliumpay.com.br/dynamic/7f47b3
e5-b5d1-49a2-b0e7-eb76105cae46520400005303986580 2BR5907RW LTDA6009Sao
Paulo62070503***6304F505"
    },
    "merchant_order_no": "M1de76cc356684c3cbc5ac59562c5bb48",
    "order_no": "TD250614138341585927556300 8",
    "endToEndId":"E250614138341563008",
    "return_code": "SUCCESS",
    "return_msg": "OK",
    "status": "PROCESSING",
    "trade_type": "pix"
}
```

对账单下载

商户通过该接口,可以下载对账单文件。

注意:

1.接口返回值不需要验签

2.对账单中涉及金额的字段单位为“元”。

对账单下载

使用HTTP协议调用对账单下载接口发起请求:

```
POST /v1/statement/download HTTP/1.1
Host: 替换支付系统域名地址/merchant-api
ContentType: application/json;charset=utf-8
Authorization: {开发者SecretKey}
nonce: {随机字符串}
timestamp: {时间戳}
sign: {签名数据}
```

请求参数

字段名	变量名	类型	必须	示例值	描述
对账单日期	bill_date	String(8)	是	20190919	下载对账单的日期，格式:yyyyMMdd
账单类型	bill_type	String(8)	否	ALL	ALL（默认值），返回所有(支付+退款)成功的交易 SUCCESS(所有支付成功的交易) REFUND(所有退款成功的交易)

返回结果

失败时返回以下字段

字段名	变量名	类型	必填	示例值	描述
网关返回码	return_code	String(16)	是	SUCCESS	错误代码，详细参见下文错误码说明 SUCCESS 成功 其他错误码均为失败
网关返回码描述	return_msg	String(128)	是	OK	当return_code为FAIL时返回信息为错误原因 ，例如签名失败，参数格式校验错误

成功时返回信息如下

成功时，数据以文本表格的方式返回，第一行为表头，后面各行为对应的字段内容，字段内容跟查询订单或退款结果一致，具体字段说明可查阅相应接口。 第一行为表头，根据请求下载的对账单类型不同而不同(由bill_type决定)，具体如下:

所有(支付+退款)成功的交易

交易时间,商户号,商户订单号,支付平台订单号,交易类型,订单类型,交易状态,交易金额(元),支付平台退款订单号,商户退款订单号,退款金额(元),手续费(元),订单描述

所有支付成功的交易

交易时间,商户号,商户订单号,支付平台订单号,交易类型,订单类型,交易状态,交易金额(元),手续费(元),订单描述

示例

交易对账单成功示例

```
***请求参数***
{
    "bill_date": "20190919"
}
***响应参数***
交易时间,商户号,商户订单号,支付平台订单号,交易类型,订单类型,交易状态,交易金额(元),交易描述,支付平台退款订单号,商户退款订单号,退款金额(元),退款状态,手续费(元),退款描述
`20190919150910,`1216,`M1de76cc356684c3cbc5ac59562c5bb48,`ND20062810495672675 12356125818,`alipay_h5,`支付,`成功,`1.00,`pix支付,`,`,`,`,`0.01,`
`20190919160910,`1216,`M1de76cc356684c3cbc5ac59562c5bb48,`ND20062810495672675 12356125818,`alipay_h5,`支
付,`,`,`,`IR200918153413509674800 3328,`MR10101010101010,`0.50,`成功,`0.01,`测试退款`
总交易单数,总交易金额(元),总退款金额(元),手续费总金额(元)
2,1.00,0.50,0.02
```

后台异步通知

后台异步通知

根据下单接口异步通知URL参数.当交易有终态时,支付平台会将**支付结果**发送到商户指定的地址.

如何验签

每一次异步通知,支付平台都会对request body整体做签名,并将签名后的结果放request header中,header名称是sign.签名算法为SHA256WithRSA,签名时会使用支付平台私钥做签名,所以商户需要用支付平台公钥来对请求进行验签.支付平台公钥可通过支付平台“开发者信息”中获取.

可靠通知规则

由于网络等原因,商户可能未接收或未正常处理支付平台异步通知,支付平台会通过一定重发机制来保证通知的可靠性。

支付平台是否重发,取决于商户收到通知后响应的结果是否为success字符串.若不是,才会按照一定机制重发,直到收到success为止。

不过支付平台重发的时间间隔也会随着重发次数逐步变长。如果支付平台收到商户的应答不是成功或超时（正常的响应是:http code 200 且 content 为 success）,支付平台认为通知失败,在36小时之内,一共发送17次,第一次发送失败之后,之后每次发送的间隔为:2s,4s,8s,16s,32s,64s,128s,256s,512s,1024s,2048s,4096s,8192s,16384s,32768s,65536s

支付平台请求参数说明

注意事项

1.我方通过HTTP POST 方式进行商户侧的异步通知URL调用

2.Content-Type是application/json;charset=UTF-8

参数	类型	描述
data	String	交易结果明细 具体参数同下单响应参数
notifyNo	String	通知事件唯一标识
timeCreated	Long	13位的时间戳,标准北京时间,时区为东八区,自1970年1月1日 0点0分0秒 以来的毫秒数
type	String	通知类型，共以下几种:CHARGE : 支付;

通知报文内容示例

支付成功通知示例

```
{
"data":{
    "order_no":"ND20110722372498476974080 0",
    "trade_type":"alipay_h5",
    "return_msg":"OK",
    "return_code":"SUCCESS",
    "merchant_order_no":"1573137443941579",
    "endToEndId":"E250614138341563008",
    "status":"SUCCESS",
    "amount":1100,
    "fee":30
},
"notifyNo":"evt_647fc95492684c55949a98d604723a35",
"timeCreated":1573137458111,
"type":"CHARGE"
}
```

商户端返回结果

返回值	说明
success	返回值必须是 **success** 字符串,**共7个字符,并且全部是小写**,否则会重新发送交易结果异步通知.

错误码

名称	描述	原因	解决方案
SECRET_KEY_IS_BLANK	SecretKey不允许为空	请求header中未传递Authorization参数	请检查后传入即可
SECRET_KEY_IS_INVALID	SecretKey错误	请求header中传递Authorization参数数值不对	核实商户后台开发者SecretKey数值
SIGN_CHECK_FAILED	签名验证失败		
ORDER_NO_DUPLICATE	订单号重复	同一笔商户订单号不能多次提交	请核实商户订单号是否重复提交
ILLEGAL_ARGUMENT	参数错误	参数格式有误或者未按规则上传	请确认请求参数是否按照接口文档要求填写
SYSTEM_ERROR	交易异常，请联系技术支持	由于某些原因造成交易处理异常，请联系支付平台平台处理	
TRADE_FAIL	交易失败	上游通道失败的具体原因	请按照要求修正之后，重新下单
TRADE_TIME_OUT_NOT_PAID	交易超时未支付	在规定时间内未支付，交易订单已关闭	该笔订单已失败，请重新下单
IP_NO_ALLOW	商户IP不在白名单列表	商户IP格式错误或者没有提前报备	商户IP需提前报备，否则请求受限

注意事项

注：本DEMO或文档中所示的默认配置参数仅供参考，接入方须视各自系统及交易情况进行相应的调整。如因采用默认配置参数导致交易异常及造成相关损失的，平台不承担相关责任。